LAB Logbook

Lab 1

```
#SID is 2520731, so the last two digits are 31.

#1)Create a vector using np.arange.
a = np.arange(31)
print(a)

#2)Change a to a 2-D array with 1 row
a1 = a.reshape(1, 31)
print(a1)

#3)Save it in another array. Print the array.
a2 = a1
print(a2)

#4)Check the shape attribute value.
print(a2.shape)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29 30]
[[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
  24 25 26 27 28 29 30]]
[[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
  24 25 26 27 28 29 30]]
(1, 31)
```

Lab 2

```python
# 1. Determine n from the last digit of the SID=2520731
n = 1

# 2. Group by relationship and hours-per-week
Group_by_relationship_hours = data.groupby(["relationship", "hours-per-week"])
Group_by_relationship_hours.size()

# 3. Reduce hours-per-week by n using a function
def reduce_hours(x):
    return x - n

data["hours-per-week-reduced"] = data["hours-per-week"].apply(reduce_hours)

data

# 4. Group by relationship and reduced hours-per-week
Group_by_relationship_reduced_hours = data.groupby(
    ["relationship", "hours-per-week-reduced"]
)

Group_by_relationship_reduced_hours.size()
```

```
                                        0
relationship  hours-per-week-reduced
Husband       12.0                      1
              39.0                      2
              44.0                      1
              79.0                      1
Not-in-family 15.0                      1
              39.0                      2
              49.0                      2
Own-child     29.0                      1
Wife          39.0                      2
dtype: int64
```

Lab 3

jupyter Week-3-Pandas-and-Seaborn_ML_in-Finance_Final-1 (1) Last Checkpoint: 3 minutes ago

File Edit View Run Kernel Settings Help Not Trusted

Code JupyterLab Python 3 (ipykernel)

[46]:
```
# Bicolour features interaction diagram
# SID = 2520731
# Last digit = 1  → Account length
# Second-to-last digit = 3 → Total day charge

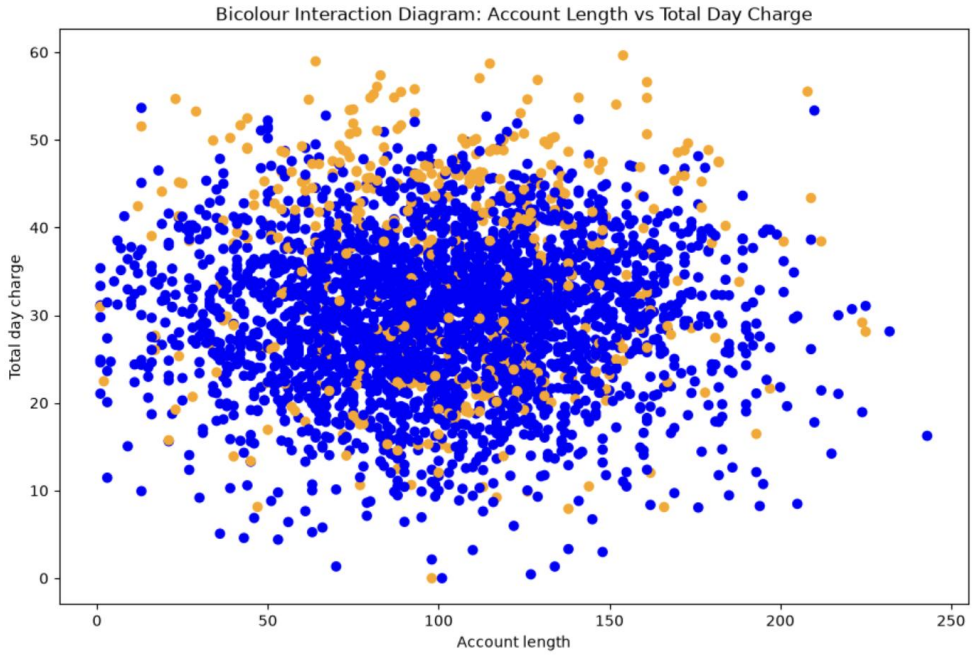
Clr = data['Churn'].map({False: 'blue', True: 'orange'})

fig = plt.figure(figsize=(11,7))

plt.scatter(data['Account length'],
            data['Total day charge'],
            color=Clr)

plt.xlabel('Account length')
plt.ylabel('Total day charge')
plt.title('Bicolour Interaction Diagram: Account Length vs Total Day Charge')

plt.show()
```



Lab 4

MLP architecture:

[70]:
```
# Create an MLP with two hidden layers based on my SID
# SID: 2520731
# First hidden layer: 731 neurons
# Second hidden layer: 366 neurons

model = keras.Sequential([
    keras.Input(shape=(X_train.shape[1],)),
    keras.layers.Dense(731, activation='relu'),
    keras.layers.Dense(366, activation='relu'),
    keras.layers.Dense(1)
])

# Display the model architecture
model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_17 (Dense)	(None, 731)	366,231
dense_18 (Dense)	(None, 366)	267,912
dense_19 (Dense)	(None, 1)	367

Total params: 634,510 (2.42 MB)

Trainable params: 634,510 (2.42 MB)

Non-trainable params: 0 (0.00 B)

MLP training code and process:

[73]:
```
#Train the MLP for 10 epochs

history = model.fit(
    X_train,
    y_train,
    batch_size=10,
    epochs=10,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/10
2640/2640 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step - loss: 0.0070 - mae: 0.0191 - val_loss: 0.0198 - val_mae: 0.1237
Epoch 2/10
2640/2640 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - loss: 2.8603e-04 - mae: 0.0128 - val_loss: 0.0086 - val_mae: 0.0819
Epoch 3/10
2640/2640 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step - loss: 1.7039e-04 - mae: 0.0097 - val_loss: 0.0040 - val_mae: 0.0568
Epoch 4/10
2640/2640 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step - loss: 1.1358e-04 - mae: 0.0082 - val_loss: 0.0037 - val_mae: 0.0555
Epoch 5/10
2640/2640 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - loss: 9.3823e-05 - mae: 0.0075 - val_loss: 0.0020 - val_mae: 0.0402
Epoch 6/10
2640/2640 ━━━━━━━━━━━━━━━━━━━━ 7s 2ms/step - loss: 7.6295e-05 - mae: 0.0065 - val_loss: 0.0010 - val_mae: 0.0279
Epoch 7/10
2640/2640 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step - loss: 6.7249e-05 - mae: 0.0062 - val_loss: 0.0013 - val_mae: 0.0320
Epoch 8/10
2640/2640 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step - loss: 6.0549e-05 - mae: 0.0059 - val_loss: 0.0012 - val_mae: 0.0313
Epoch 9/10
2640/2640 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step - loss: 6.3413e-05 - mae: 0.0060 - val_loss: 5.9955e-04 - val_mae: 0.0209
Epoch 10/10
2640/2640 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step - loss: 5.6988e-05 - mae: 0.0057 - val_loss: 2.5761e-04 - val_mae: 0.0127
```

Resulting MAE on the test dataset:

[74]:
```
#Evaluate the model on the test dataset

mse, mae = model.evaluate(X_test, y_test, verbose=0)

print("Mean absolute error: %.5f" % mae)
print("Mean squared error: %.5f" % mse)
```

```
Mean absolute error: 0.03683
Mean squared error: 0.00174
```

[75]:
```
#Compare your MAE with the practical model

practical_mae = 0.02932

print("Practical model MAE:", practical_mae)
print("My MLP test MAE:", round(mae, 5))

if mae < practical_mae:
    print("My MLP achieved a lower test MAE than the practical model.")
elif mae > practical_mae:
    print("My MLP achieved a higher test MAE than the practical model.")
else:
    print("Both models achieved the same test MAE.")
```

```
Practical model MAE: 0.02932
My MLP test MAE: 0.03683
My MLP achieved a higher test MAE than the practical model.
```

Lab 5

[39]:
```
mae_practical = mae      # keep the practical session's test MAE for comparison

model_lab = keras.Sequential([

    keras.layers.Input(shape=(50, 5)),

    keras.layers.Conv1D(50, 5, padding='same', activation=tf.nn.relu, kernel_initializer="normal"),

    keras.layers.MaxPooling1D(7),

    keras.layers.Conv1D(100, 5, padding='same', activation=tf.nn.relu, kernel_initializer="normal"),

    keras.layers.GlobalMaxPooling1D(),

    keras.layers.Dense(25, activation=tf.nn.relu, kernel_initializer="normal"),

    keras.layers.Dense(2)
])

model_lab.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv1d_2 (Conv1D)	(None, 50, 50)	1,300
max_pooling1d_1 (MaxPooling1D)	(None, 7, 50)	0
conv1d_3 (Conv1D)	(None, 7, 100)	25,100
global_max_pooling1d_1 (GlobalMaxPooling1D)	(None, 100)	0
dense_2 (Dense)	(None, 25)	2,525
dense_3 (Dense)	(None, 2)	52

Total params: 28,977 (113.19 KB)

Trainable params: 28,977 (113.19 KB)

Non-trainable params: 0 (0.00 B)

[40]:
```
model_lab.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

[41]:
```
history_lab = model_lab.fit(X_train, y_train, batch_size=50, epochs=epochs_lab, validation_split=0.2, verbose=1)
```

```
Epoch 1/4
3520/3520 ━━━━━━━━━━━━━━━━━━━━ 5s 1ms/step - loss: 0.0022 - mae: 0.0261 - val_loss: 0.0011 - val_mae: 0.0227
Epoch 2/4
3520/3520 ━━━━━━━━━━━━━━━━━━━━ 4s 1ms/step - loss: 7.4831e-04 - mae: 0.0186 - val_loss: 9.0352e-04 - val_mae: 0.0199
Epoch 3/4
3520/3520 ━━━━━━━━━━━━━━━━━━━━ 4s 1ms/step - loss: 7.2002e-04 - mae: 0.0181 - val_loss: 8.6218e-04 - val_mae: 0.0194
Epoch 4/4
3520/3520 ━━━━━━━━━━━━━━━━━━━━ 4s 1ms/step - loss: 7.0874e-04 - mae: 0.0179 - val_loss: 8.9643e-04 - val_mae: 0.0202
```

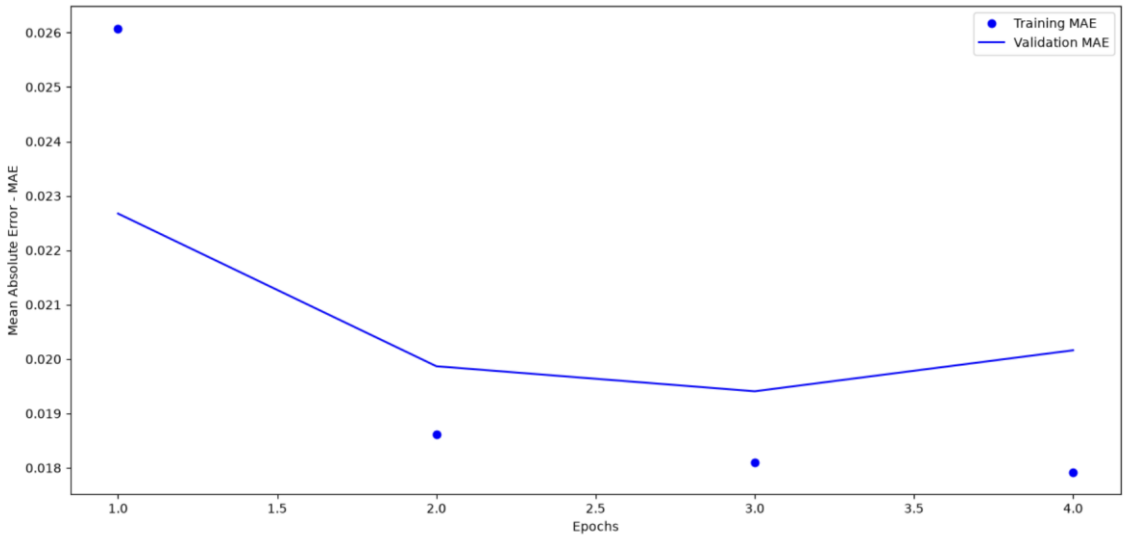
[42]:
```
mse_lab, mae_lab = model_lab.evaluate(X_test, y_test, verbose=1)
print("Lab CNN test MAE:        %.5f" % mae_lab)
print("Practical CNN test MAE:  %.5f" % mae_practical)
```

```
936/936 ━━━━━━━━━━━━━━━━━━━━ 0s 324us/step - loss: 6.5530e-04 - mae: 0.0178
Lab CNN test MAE:        0.01784
Practical CNN test MAE:  0.01639
```

[43]:
```
history_dict = history_lab.history

mean_absolute_error_values = history_dict['mae']
val_mean_absolute_error_values = history_dict['val_mae']

epochs_range = range(1, len(mean_absolute_error_values) + 1)
plt.figure(num=1, figsize=(15,7))
plt.plot(epochs_range, mean_absolute_error_values, 'bo', label='Training MAE')
plt.plot(epochs_range, val_mean_absolute_error_values, 'b', label='Validation MAE')
plt.xlabel('Epochs')
plt.ylabel('Mean Absolute Error - MAE')
plt.legend()
plt.show()
```



Lab 6

```
start  = 20731
period = 731

subset = data.iloc[start:start + period]

fig = px.line(
    subset,
    x='Local_time_T',
    y=['High_Bid', 'Low_Bid'],
    markers=True,
    title='GOLD (rows 20731 to 21462, 731 minutes)',
)
fig.update_layout(xaxis_title='Date', yaxis_title='Price', legend_title_text='')
fig.show()
```



Lab 7

LSTM architecture using model.summary() :-

```
# Model architecture: shows the LSTM layer with 41 units and the Dense(2) output layer,
# along with the number of trainable parameters.
# Fewer units than the practical session (100) means fewer parameters and faster training.

print(model.summary())
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 41)	9,840
dense_1 (Dense)	(None, 2)	84

Total params: 9,924 (38.77 KB)

Trainable params: 9,924 (38.77 KB)

Non-trainable params: 0 (0.00 B)

None

training process :-

```
# Training
# CHANGED: epochs 20 -> 10 (maximum number of passes over the training data)
# Unchanged: batch_size=20, validation_split=0.1 (10% of training data used for validation)
# Early stopping may stop training before 10 epochs if val_loss stops improving for 3 epochs.

history = model.fit(X_train, y_train, batch_size=20, epochs=10, validation_split=0.1, shuffle=True, verbose=1, callbacks=[es, mc])
```

```
Epoch 1/10
1203/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0139 - mae: 0.0180
Epoch 1: val_loss improved from None to 0.00003, saving model to best_model_LSTM_GOLD_SID.keras

Epoch 1: finished saving model to best_model_LSTM_GOLD_SID.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 8s 6ms/step - loss: 0.0137 - mae: 0.0179 - val_loss: 3.1060e-05 - val_mae: 0.0036
Epoch 2/10
1209/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 2.5012e-05 - mae: 0.0035
Epoch 2: val_loss improved from 0.00003 to 0.00003, saving model to best_model_LSTM_GOLD_SID.keras

Epoch 2: finished saving model to best_model_LSTM_GOLD_SID.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - loss: 2.5004e-05 - mae: 0.0035 - val_loss: 2.6865e-05 - val_mae: 0.0031
Epoch 3/10
1204/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 2.0690e-05 - mae: 0.0032
Epoch 3: val_loss improved from 0.00003 to 0.00002, saving model to best_model_LSTM_GOLD_SID.keras

Epoch 3: finished saving model to best_model_LSTM_GOLD_SID.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - loss: 2.0655e-05 - mae: 0.0032 - val_loss: 2.1099e-05 - val_mae: 0.0027
Epoch 4/10
1211/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 1.9922e-05 - mae: 0.0033
Epoch 4: val_loss improved from 0.00002 to 0.00002, saving model to best_model_LSTM_GOLD_SID.keras

Epoch 4: finished saving model to best_model_LSTM_GOLD_SID.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - loss: 1.9922e-05 - mae: 0.0033 - val_loss: 1.9301e-05 - val_mae: 0.0028
Epoch 5/10
1205/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 1.9381e-05 - mae: 0.0034
Epoch 5: val_loss did not improve from 0.00002
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - loss: 1.9422e-05 - mae: 0.0034 - val_loss: 2.0105e-05 - val_mae: 0.0039
Epoch 6/10
1206/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 1.9247e-05 - mae: 0.0035
Epoch 6: val_loss improved from 0.00002 to 0.00001, saving model to best_model_LSTM_GOLD_SID.keras

Epoch 6: finished saving model to best_model_LSTM_GOLD_SID.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 7s 6ms/step - loss: 1.9225e-05 - mae: 0.0035 - val_loss: 7.0955e-06 - val_mae: 0.0018
Epoch 7/10
1209/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 1.5835e-05 - mae: 0.0031
Epoch 7: val_loss did not improve from 0.00001
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - loss: 1.5807e-05 - mae: 0.0031 - val_loss: 3.8059e-05 - val_mae: 0.0058
Epoch 8/10
1211/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 1.5821e-05 - mae: 0.0032
Epoch 8: val_loss improved from 0.00001 to 0.00000, saving model to best_model_LSTM_GOLD_SID.keras

Epoch 8: finished saving model to best_model_LSTM_GOLD_SID.keras
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 7s 6ms/step - loss: 1.5802e-05 - mae: 0.0032 - val_loss: 2.9746e-06 - val_mae: 0.0013
Epoch 9/10
1205/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 1.2342e-05 - mae: 0.0028
Epoch 9: val_loss did not improve from 0.00000
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 7s 6ms/step - loss: 1.2287e-05 - mae: 0.0028 - val_loss: 6.9788e-06 - val_mae: 0.0023
Epoch 10/10
1208/1213 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 1.0895e-05 - mae: 0.0026
Epoch 10: val_loss did not improve from 0.00000
1213/1213 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - loss: 1.0896e-05 - mae: 0.0026 - val_loss: 2.4890e-05 - val_mae: 0.0048
```

the resulting test MSE & MAE :-

```
# Evaluate on the test set (the last 10% of the data, never seen during training).
# scores[0] = test MSE, scores[1] = test MAE
# Note: prices were divided by 1000 in normalisation, so an error of 0.001 is about $1.

scores = LSTM_saved_best_model.evaluate(X_test, y_test, verbose=1)

print("Mean squared error (mse): %.9f " % (scores[0]))
print("Mean absolute error (mae): %.9f " % (scores[1]))
```

```
94/94 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 2.0147e-06 - mae: 0.0011
Mean squared error (mse): 0.000002015
Mean absolute error (mae): 0.001139615
```

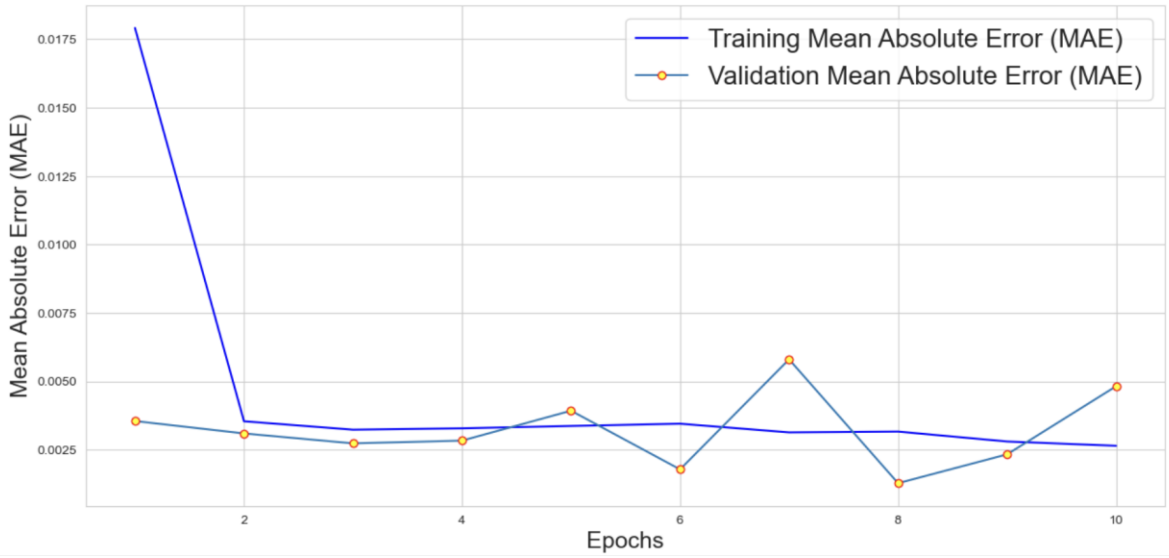
MAE detailed graph :-

```
# Detailed MAE graph
# Blue line   = training MAE per epoch
# Yellow dots = validation MAE per epoch
# If validation MAE flattens or rises while training MAE keeps falling, the model is
# starting to overfit, which is what early stopping is meant to catch.

history_dict = history.history

mae_values = history_dict['mae']
val_mae_values = history_dict['val_mae']

epochs = range(1, len(mae_values) + 1)
plt.figure(num=1, figsize=(15,7))
plt.plot(epochs, mae_values, 'b', label='Training Mean Absolute Error (MAE)')
plt.plot(epochs, val_mae_values, marker='o', markeredgecolor='red', markerfacecolor='yellow', label='Validation Mean Absolute Error (MAE)')
plt.xlabel('Epochs', size=18)
plt.ylabel('Mean Absolute Error (MAE)', size=18)
plt.legend()
plt.show()
```



Lab 8

Lab 9

Lab 10

Lab 11

Lab 12